

Jomhoor Encryption and Cryptography Technical Description

Document date: June 10, 2026

Application: Jomhoor

iOS bundle identifier: org.jomhoor.app

Application version reviewed: 0.5.34

Platform: iOS

1. Application Purpose

Jomhoor is a privacy-preserving digital identity, wallet, and civic participation application. The application enables users to:

- Create or import a locally controlled cryptographic wallet.
- Verify identity using supported electronic passports and national identity documents.
- Generate zero-knowledge proofs concerning verified identity attributes.
- Authenticate to compatible services.
- Participate in identity-gated proposals and polls.
- Sign messages and blockchain transactions.

Identity documents and biometric information are processed primarily on the user's device. Zero-knowledge proofs are used to demonstrate identity-derived eligibility without disclosing unnecessary underlying identity information.

The application is not a general-purpose encryption product. It does not offer users arbitrary file encryption, encrypted messaging, VPN functionality, or user-selectable cryptographic algorithms.

2. Summary of Cryptographic Functions

The application contains cryptography for the following purposes:

1. Secure NFC communication with electronic identity documents.
2. Authentication and integrity verification of electronic-document data.
3. Generation and use of locally controlled wallet keys.
4. Digital signing of authentication challenges, messages, and blockchain transactions.
5. Generation of privacy-preserving commitments and nullifiers.
6. Local generation of zero-knowledge identity and eligibility proofs.
7. Secure storage of private keys using the iOS Keychain.
8. HTTPS/TLS communication with application services and blockchain endpoints.
9. Application-integrity attestation using Apple App Attest.

3. Electronic Passport NFC Cryptography

Jomhoor reads supported ICAO 9303-compatible electronic passports using NFC. The application establishes a temporary secure channel with the document chip and validates the authenticity and integrity of document data.

3.1 Protocols

The electronic-passport implementation contains support for:

- Basic Access Control (BAC)
- Password Authenticated Connection Establishment (PACE)
- ICAO secure messaging
- Passive Authentication
- Active Authentication
- Chip Authentication

3.2 Symmetric Encryption

Algorithm	Modes or variants	Purpose
Triple DES	Two-key EDE, CBC	BAC mutual authentication and ICAO secure messaging for documents using legacy BAC or PACE profiles
AES	128, 192, and 256-bit keys; CBC and ECB operations	PACE and ICAO secure messaging for supported electronic documents

The application derives or negotiates temporary session encryption keys for a single NFC session. These keys protect APDU commands and responses exchanged between the application and the identity-document chip.

AES-ECB is used only for the protocol-defined derivation of an initialization vector in ICAO secure messaging. It is not used to encrypt arbitrary user data.

3.3 Message Authentication

Algorithm	Purpose
ISO/IEC 9797-1 MAC Algorithm 3, also called Retail MAC	Authentication and integrity protection of BAC and legacy secure-messaging data
AES-CMAC	Authentication and integrity protection in supported PACE and AES secure-messaging profiles
DES-based CBC MAC operations	Protocol-required authentication for legacy electronic-document profiles

3.4 Key Agreement and Key Derivation

Algorithm or protocol	Purpose
Diffie-Hellman	Key agreement for supported PACE and Chip Authentication profiles
Elliptic Curve Diffie-Hellman	Key agreement for supported PACE and Chip Authentication profiles
SHA-1 and SHA-256 based ICAO key derivation	Derivation of BAC, PACE, and secure-messaging keys as required by the document profile

BAC access keys are derived from machine-readable-zone information. PACE and Chip Authentication use protocol-defined ephemeral key agreement. Derived and negotiated session keys are not retained as long-term user keys.

3.5 Hash and Signature Validation

The electronic-document implementation processes the following algorithms:

- SHA-1
- SHA-224

- SHA-256
- SHA-384
- SHA-512
- RSA PKCS#1 v1.5 signatures
- RSA-PSS signatures
- ECDSA signatures
- X.509 certificates
- CMS/PKCS#7 signed data

These algorithms are used to:

- Compare document data-group hashes with the signed Security Object Document.
- Verify document-signing certificates and signatures.
- Verify supported active-authentication signatures.
- Process supported chip-authentication public keys.
- Detect modification of electronic-document data.

3.6 Supported Elliptic Curves

Document-processing code contains support for:

- NIST P-192
- NIST P-224
- NIST P-256
- NIST P-384
- NIST P-521
- Brainpool P-256
- Brainpool P-320
- Brainpool P-384
- Brainpool P-512

The exact curve and algorithm used during a document session are selected from the security information contained in the electronic document.

3.7 Implementations

The native iOS passport reader uses NFCPassportReader and a bundled OpenSSL 3.5.5-based library for supported cryptographic and certificate operations.

The application also contains a JavaScript ICAO BAC implementation using SHA-1, Triple DES, ISO/IEC 9797-1 MAC Algorithm 3, and cryptographically secure random nonces. This implementation provides the same protocol-limited electronic-document access function and is not a general-purpose encryption interface.

4. Wallet and Blockchain Cryptography

4.1 Wallet Private Key

The application generates a 256-bit private value locally or accepts a private key imported by the user. The private key is stored on iOS using Keychain-backed secure storage.

The application does not provide server-side private-key escrow or recovery. The private key is not intentionally transmitted to Jomhoor application services.

4.2 Baby Jubjub Wallet

The identity wallet uses:

- Baby Jubjub elliptic-curve scalar multiplication
- Poseidon hashing over the BN254 field
- A Poseidon-based, Baby Jubjub EdDSA-compatible challenge signature

These operations are used to:

- Derive the wallet public key.
- Derive a privacy-preserving wallet identifier.
- Generate authentication challenge signatures.
- Generate commitments and nullifiers used by identity and participation protocols.

The challenge-signature construction uses a deterministic Poseidon-derived nonce and Baby Jubjub group operations. It provides authentication and does not encrypt user content.

4.3 EVM-Compatible Wallet

The application derives an EVM-compatible wallet from locally held private-key material and uses:

- ECDSA over secp256k1
- Keccak-256
- Ethereum-compatible address derivation
- Ethereum-compatible transaction signing
- Ethereum-compatible message signing

Signing is performed locally on the device. The resulting signatures and transactions may be transmitted to blockchain RPC or relay services.

5. Zero-Knowledge Proof Cryptography

Jomhoor generates zero-knowledge proofs locally for identity registration, authentication, and eligibility checks.

The application contains:

- Groth16 proof generation using Rapidsnark
- Noir PLONK/UltraPLONK proof generation using the Swoirenborg backend
- BN254 finite-field and pairing-based proof operations
- Poseidon-based commitments, nullifiers, and sparse Merkle tree values

Zero-knowledge proof generation is used to prove statements such as:

- Possession of a wallet private key.
- Validity of supported identity-document data and signatures.
- Possession of a previously registered identity.
- Satisfaction of selected age, citizenship, or eligibility requirements.
- Uniqueness of participation through a context-specific nullifier.

The proof systems do not encrypt arbitrary files or communications. They produce proofs and public signals that allow a verifier to validate a statement without receiving all private witness values.

Circuit bytecode, proving keys, and structured reference strings may be downloaded over HTTPS or bundled with the application. These public proving artifacts do not contain a user's private key.

6. Standard Hash Functions Used Outside NFC

The application implements or invokes:

- SHA-1
- SHA-224
- SHA-256
- SHA-384
- SHA-512
- Keccak-256
- Poseidon

The SHA family is used for electronic-document processing, certificate and signature processing, and circuit input preparation. Keccak-256 is used for EVM-compatible addresses and encoded blockchain data. Poseidon is used for zero-knowledge-friendly commitments, wallet identifiers, signatures, nullifiers, and sparse Merkle tree values.

7. Apple Operating-System Security Functions

The application also uses cryptography and security services supplied by iOS:

- iOS Keychain through Expo SecureStore for private-key storage
- HTTPS/TLS through Apple networking facilities
- Secure random-number generation exposed through the application runtime
- Local Authentication and Face ID for local access control
- DeviceCheck App Attest for application-integrity attestation

The application does not implement a custom TLS protocol for ordinary API communication.

8. Key and Sensitive-Data Management

- Wallet private keys are generated locally or imported by the user.
- Wallet private keys are stored using iOS Keychain-backed secure storage.
- Wallet private keys are used locally for wallet derivation, signatures, and zero-knowledge witnesses.
- Electronic-document BAC, PACE, and secure-messaging keys are temporary session keys.
- Electronic-document session keys are not retained as permanent credentials.
- NFC document data is processed for identity verification and proof generation.
- Zero-knowledge witness values are processed locally during proof generation.
- Proving keys and trusted setup data are public cryptographic parameters and contain no user private keys.
- HTTPS/TLS protects application network traffic in transit.

9. Data Protected or Authenticated

The cryptographic functions protect or authenticate:

- Wallet ownership
- Authentication challenges
- Blockchain messages and transactions
- NFC communication with supported electronic identity documents

- Integrity and authenticity of electronic-document data
- Privacy-preserving identity commitments
- Context-specific participation nullifiers
- Zero-knowledge claims about identity and eligibility
- Private keys stored in the iOS Keychain
- Network communication transported over HTTPS/TLS

10. Cryptographic Components

The application includes or invokes the following relevant components:

- OpenSSL 3.5.5 local iOS framework
- NFCPassportReader
- @iden3/js-crypto
- @noble/curves
- @noble/hashes
- ethers
- crypto-browserify
- create-hash
- des.js
- Rapidsnark
- Noir/Swoirenborg
- Native witness-calculation libraries
- Expo SecureStore

Some dependencies are general-purpose cryptographic libraries. The inclusion of a library does not mean that every algorithm offered by that library is used. The algorithms described in this document are those identified in the application's runtime paths and electronic-document implementation.

11. Functional Boundaries

Jomhoor does not provide:

- General-purpose file or disk encryption
- Encrypted person-to-person messaging
- VPN, tunneling, or proxy services
- Cryptographic key escrow
- Enterprise key-management services
- User-defined cryptographic algorithms
- Cryptanalysis functionality

Cryptography is integrated into the application's identity-document, wallet, authentication, blockchain, secure-storage, and privacy-preserving proof functions.